

SOFTWARE ENGINEERING MINI PROJECT

**RETAIL INVENTORY AND STOCK MANAGEMENT
SYSTEM**

Software Architecture & Design Specification

Deliverable Part-1 | Team E10

Team Members

Name	SRN
Malavika A	PES1UG24CS257
Manoj Kumar	PES1UG24CS264
Mayank M	PES1UG24CS266
LG Vivyn Krishnaa	PES1UG24CS248

1. Introduction

1.1 Purpose

This document describes the software architecture and detailed design of the **Retail Inventory and Stock Management System**. It explains how the requirements captured in the Software Requirements Specification are realised as a set of components, interfaces, data structures and interactions, and it provides enough detail for the team to implement the system and for the Test Plan to be traced back to design elements. It translates the Phase 1 requirements into components, interfaces, data formats and behaviour that the team will implement in C++ and verify through the Test Plan.

1.2 Scope

RISMS is a menu-driven C++ console application that lets an authorised Inventory User maintain product records, search and view products, increase or decrease stock, check availability, review stock movement history and view an inventory summary with low-stock items. All data is persisted in local files. A graphical or web interface, a database server, barcode scanners, point-of-sale integration and multiple user roles are outside the scope of Part-1.

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
RISMS	Retail Inventory and Stock Management System
SRS	Software Requirements Specification (Phase 1 document)
FR / VFR / NFR	Functional / Verifiable Functional / Non-Functional Requirement
SR	Security Requirement
UC	Use Case (UC01–UC11 as in the SRS)
Stock-in / Stock-out	An adjustment that increases / decreases a product's quantity
Ledger	The append-only stock movement history file (stock_ledger.csv)
Low-stock threshold	Configured quantity at or below which a product is flagged (VFR09)
Atomic write	Writing to a temporary file and renaming it over the original, so a crash never leaves a half-written file
Repository	Class that hides file formats and I/O from the business logic

1.4 References

- SRS – Retail Inventory and Stock Management System, Team E10, Phase 1 deliverable (FR01–FR14, VFR01–VFR09, UC01–UC11).
- Test Plan – Retail Inventory and Stock Management System, Team E10, Part-1 deliverable.
- IEEE Std 1016-2009, Systems Design – Software Design Descriptions.
- ISO/IEC/IEEE 42010:2011, Architecture description.
- SEI CERT C++ Coding Standard .

1.5 Document Overview

Section 2 presents the architecture: drivers, chosen pattern, component diagram and descriptions, deployment, data design, traceability to requirements and the security architecture.

Section 3 presents the detailed design: UML sequence diagrams, API (module interface) design, error handling and key algorithms. Section 4 lists constraints and assumptions and Section 5 records the design-to-test hooks used by the Test Plan.

1.6 Assumptions on Non-Functional and Security Requirements

The Phase 1 SRS lists functional requirements only. The following non-functional requirements and security requirements are assumed as the architectural drivers used throughout this document. If the numbering or thresholds in the final SRS differ, the traceability tables in Section 2.7 must be updated accordingly.

ID	Requirement (measurable)
NFR01 Performance	Search by product ID shall return in under 1 second with 10,000 stored products.
NFR02 Reliability	No product or ledger record shall be lost or corrupted if the program is terminated during a save.
NFR03 Usability	Every menu operation shall complete in at most 5 prompts, and every error message shall state the reason.
NFR04 Portability	The program shall build as C++17 with g++ 9+ or MSVC 2019+ and run on Windows and Linux without code changes.
NFR05 Maintainability	Each service and repository class shall be unit-testable without the console layer.
SR01	Passwords shall never be stored in plain text; only a salted SHA-256 hash shall be stored.
SR02	Login shall be locked for 60 seconds after 3 consecutive failed attempts.
SR03	No menu option other than Login and Exit shall be reachable without an authenticated session.
SR04	All user input shall be length-limited and type-checked before use.
SR05	Stock history shall be append-only and each entry shall record the acting user and time.
SR06	A failed or interrupted save shall not corrupt existing data files.
SR07	Data files shall be readable and writable only by the operating-system account running RISMS.

Stakeholders and Concerns

Stakeholder	Primary design concerns
Inventory User (A01)	Simple menus, clear messages, accurate stock figures, no lost data
Store owner	Correct low-stock reporting, audit trail of stock changes, restricted access
Developers	Modular code, clear interfaces, independent module testing
Testers / evaluators	Every requirement traceable to a component and a test case

2. Architectural Design

2.1 Architectural Drivers

The following requirements influence the architecture the most:

- **Correct stock arithmetic (VFR05, VFR06):** quantity must change only through controlled stock-in and stock-out operations, never through a general product edit.
- **Traceable history (FR09, VFR07, SR05):** every successful adjustment must reach the ledger, so product save and ledger append must be coordinated in one place.
- **Persistence without a database (FR13, FR14, VFR08, NFR02):** files must survive restarts and crashes, so all file I/O is concentrated in one component that performs atomic writes.
- **Modifiability (NFR05):** the SRS schedule analysis splits the work into product records, stock adjustment, lookup and history/report modules that are developed and tested independently, so module boundaries must be explicit.

2.2 Architectural Pattern

RISMS uses a strict three-layer architecture with the Repository pattern at the data layer. Each layer calls only the layer directly below it. The presentation layer never touches a file, and the service layer never knows a file name or format.

Layer	Responsibility	Modules
Presentation layer	Show menus, read and parse input, print results and error messages, check that a session exists before dispatching a command.	MenuController, ConsoleIO
Business logic layer (services)	Business rules: authentication, validation, stock arithmetic, ledger entries, low-stock detection, reports.	AuthService, ProductService, StockService, HistoryService, ReportService, Validator
Data access layer (repositories)	Load, cache, save and query records; hide file formats; atomic writes and appends.	UserRepository, ProductRepository, MovementRepository, ConfigRepository
Persistent storage	CSV and text files in a local data/ directory.	Local file system

Why this pattern

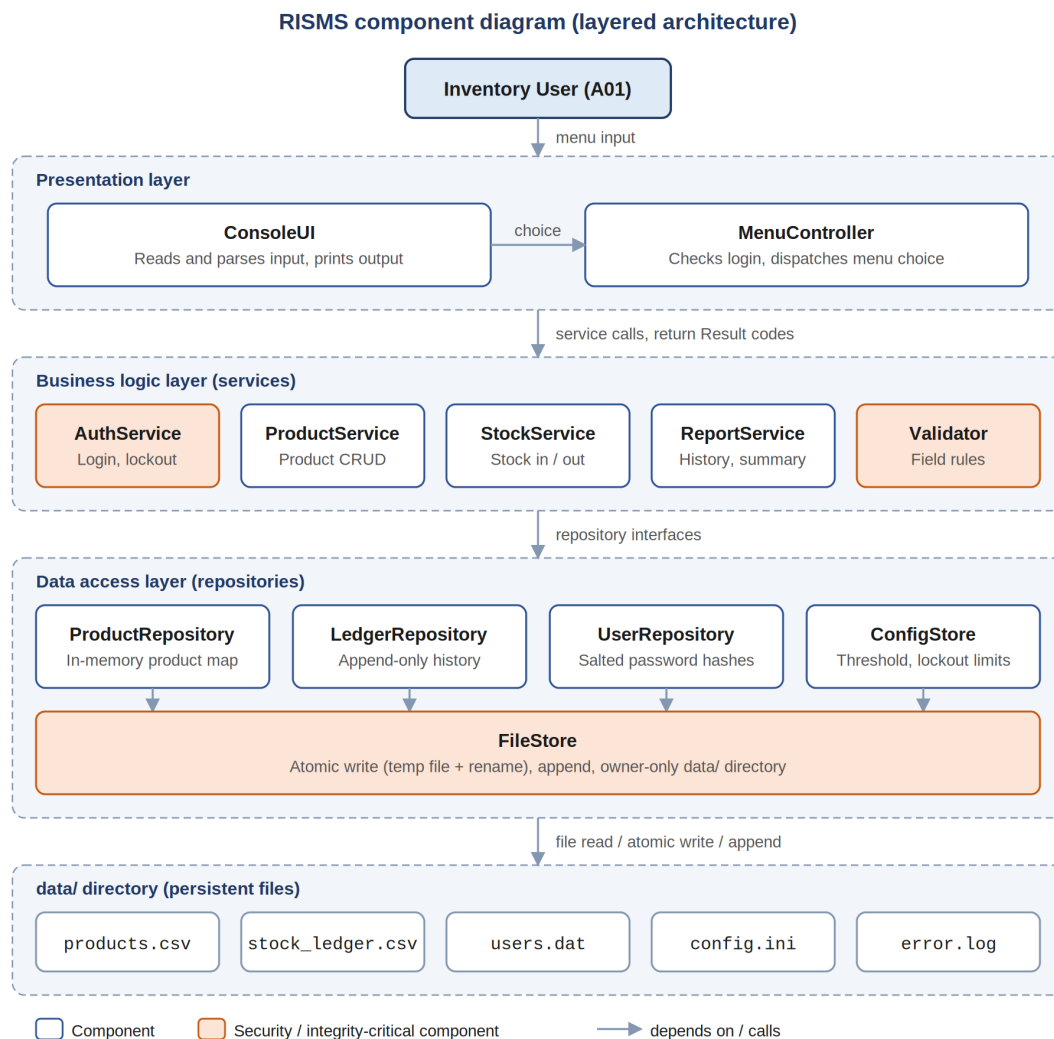
- **Separation of concerns:** the rule “stock-out cannot exceed available quantity” lives only in StockService, so it is tested once without driving the console (NFR05).
- **Testability:** services receive repository interfaces in their constructors, so unit tests pass in-memory fakes instead of real files.
- **Replaceable persistence:** moving from CSV files to SQLite later means one new repository class, not a rewrite of the business logic.
- **Right-sized for the project:** a single process with an in-memory cache needs no threads, sockets or external libraries.

Alternatives considered

Alternative	Reason for rejection
Single-file procedural program (functions and global arrays)	Fastest to start, but validation, file I/O and menu code become entangled, which makes VFR tests hard to isolate.
Model–View–Controller	Adds little over layering for a console application with one view; the controller role is already covered by MenuController.
Client–server with a database	Out of scope per the SRS (file handling, no external database) and adds installation cost.

2.3 Component Diagram

Figure 1 shows the components of RISMS grouped by layer and the dependencies between them. Orange components are security- and integrity-critical.



2.4 Component Descriptions

Component	Responsibility	Provided interface	Requirements
ConsoleUI	Prints menus, tables and messages; reads lines with <code>std::getline</code> ; parses numbers safely.	<code>readText()</code> , <code>readInt()</code> , <code>readDouble()</code> , <code>show()</code>	NFR03, SR04

Component	Responsibility	Provided interface	Requirements
MenuController	Runs the login screen and main menu loop; blocks every command until a session exists; maps each choice to a service call.	run()	FR01, SR03
AuthService	Verifies credentials against salted hashes, counts failures, applies lockout, holds the current user.	login(), logout(), isAuthenticated(), currentUser()	FR01, SR01, SR02
ProductService	Add, update, remove, search and list products; rejects duplicate IDs.	addProduct(), updateProduct(), removeProduct(), findById(), searchByName(), listAll()	FR02–FR06, FR11, VFR01–VFR04
StockService	Stock-in and stock-out with arithmetic and availability checks; one ledger entry per successful change.	stockIn(), stockOut(), getQuantity()	FR07–FR09, VFR05–VFR07
ReportService	Stock history per product; inventory summary with totals and low-stock list.	historyFor(), inventorySummary()	FR10, FR12, VFR09
Validator	Single home for field rules: required fields, lengths, allowed characters, non-negative numbers.	validateProduct(), validateQty()	VFR02, SR04
ProductRepository	Loads products.csv into an unordered_map; saves the full file on change.	load(), get(), exists(), all(), put(), erase(), save()	FR13, FR14, NFR01
LedgerRepository	Appends movements to stock_ledger.csv; reads history for a product; no update or delete.	append(), forProduct(), nextId()	FR09, FR10, SR05
UserRepository	Reads and writes users.dat (user ID, salt, hash).	find(), create()	SR01
ConfigStore	Reads config.ini; falls back to defaults if a key is missing.	lowStockThreshold(), maxAttempts(), lockoutSeconds()	VFR09, SR02
FileStore	All file I/O: creates data/ with owner-only permissions, reads lines, atomic rewrite, append.	ensureDataDir(), readLines(), writeAtomic(), appendLine()	FR13, VFR08, NFR02, SR06, SR07

2.5 Deployment View

RISMS is deployed on a single node; there are no servers or network links:

- **Store PC:** Windows 10+ or Linux machine able to run a console program.
- **Executable:** risms (Linux) or risms.exe (Windows), containing all three layers, built with `g++ -std=c++17 -O2 src/*.cpp -o risms` or the equivalent CMake target.
- **Data directory:** data/ next to the executable, created on first run with owner-only permissions, holding products.csv, stock_ledger.csv, users.dat, config.ini and error.log.

2.6 Data Design

The principal persistent entities and their key attributes are shown below. Attributes marked (U) are unique, (I) are indexed in memory.

Entity (file)	Key attributes	Notes
Product (products.csv)	productId (U, I), name, category, unitPrice, quantity	ID 1–10 letters/digits; name 1–50 chars; price and quantity ≥ 0 . Whole file rewritten atomically on change.
StockMovement (stock_ledger.csv)	movementId (U), productId, type {IN, OUT}, quantityChange, resultingQty, timestamp, userId, reason	Append-only (SR05). quantityChange > 0 ; direction given by type. History kept after a product is removed.
User (users.dat)	userId (U), saltHex, hashHex	Password never stored in plain text (SR01). Initial account created on first run.
Config (config.ini)	low_stock_threshold (default 10), max_login_attempts (3), lockout_seconds (60)	Defaults used if a key is missing.
ErrorLog (error.log)	timestamp, level {INFO, WARN, ERROR}, component, message	Security events and I/O errors; never shown to the user.

2.7 Traceability to Requirements

Table 2.7-a maps every functional and verifiable requirement in the SRS to the use case, the responsible component and the operation that realises it. Table 2.7-b does the same for non-functional and security requirements.

Table 2.7-a: Functional requirements

Requirement	Use case	Component(s)	Operation / design element
FR01	UC01	MenuController, AuthService	AuthService::login(); session check before every command
FR02, VFR01, VFR02	UC02	ProductService, Validator	addProduct() $\rightarrow$ DUPLICATE_ID / INVALID_INPUT
FR03	UC03	ProductService, Validator	updateProduct() (quantity not editable here)
FR04	UC04	ProductService	removeProduct() with Y/N confirmation
FR05, VFR03, VFR04	UC05	ProductService	findById(), searchByName() $\rightarrow$ NOT_FOUND
FR06	UC06	ProductService, ConsoleUI	findById() and detail view
FR07, VFR05, VFR06	UC07	StockService	stockIn(), stockOut() $\rightarrow$ INSUFFICIENT_STOCK
FR08	UC08	StockService	getQuantity()
FR09, VFR07	UC07	StockService, LedgerRepository	LedgerRepository::append() after every successful adjustment
FR10	UC09	ReportService	historyFor(), newest first
FR11	UC10	ProductService	listAll(), sorted by product ID
FR12, VFR09	UC11	ReportService, ConfigStore	inventorySummary(): quantity $\leq$ threshold
FR13	UC02, UC07	Repositories, FileStore	save(), append(), writeAtomic()
FR14, VFR08	UC01	ProductRepository, LedgerRepository	load() at start-up; atomic write guarantees intact files

Table 2.7-b: Non-functional and security requirements

Requirement	Design decision that satisfies it
NFR01	Products cached in an unordered_map keyed on productId; the file is read once at start-up.
NFR02	FileStore::writeAtomic() (temp file, flush, rename); product saved before ledger append with rollback on failure.
NFR03	Numbered menu of 10 options; each operation asks for at most 5 fields; error catalogue gives one-line reasons.
NFR04	Standard C++17 library only (plus header-only picosha2); std::filesystem for paths and permissions.
NFR05	Services depend on repository interfaces injected through constructors; unit tests use in-memory fakes.
SR01	UserRepository stores SHA-256(salt + password) with a 16-byte random salt; constant-time comparison.
SR02	AuthService keeps failure count and lock-until time; values read from config.ini.
SR03	MenuController calls isAuthenticated() before dispatching any option.
SR04	std::getline into std::string (no fixed buffers); std::stoi/std::stod inside try/catch; Validator caps lengths and rejects commas and newlines.
SR05	LedgerRepository exposes append and read methods only; entries carry userId and timestamp.
SR06	writeAtomic() for products.csv and users.dat; appendLine() with flush for the ledger.
SR07	FileStore::ensureDataDir() creates data/ with permissions 0700 (Linux) or current-user-only ACL (Windows).

2.8 Security Architecture

2.8.1 Security objectives

Objective	Description
SO1 Confidentiality and access control	Only an authenticated Inventory User can view or change inventory data; credentials are never stored or shown in plain text.
SO2 Integrity	Product quantities and history change only through validated service operations, and data files are never left half-written.
SO3 Accountability	Every stock change can be traced to a user, a time and a reason.

2.8.2 Security controls by layer

Layer	Control	Objective / SR
Presentation	Input read as whole lines into std::string; numeric parsing in try/catch; no raw file paths or exception text printed.	SO2 / SR04
Session	Login gate before the main menu; session cleared on logout and on exit.	SO1 / SR03
Authentication	Salted SHA-256 hashes, constant-time compare, 3-attempt lockout for 60 s, generic "incorrect user ID or password" message.	SO1 / SR01, SR02
Business logic	Every service re-validates arguments; quantity changed only via stockIn/stockOut; stock-out bounded by available quantity.	SO2 / SR04
Data access	Append-only ledger; atomic rewrite of products and users; malformed lines skipped and logged, never guessed.	SO2, SO3 / SR05, SR06
Storage	data/ directory restricted to the owning OS account.	SO1 / SR07

Layer	Control	Objective / SR
Audit	error.log records login success, failure, lockout, logout and I/O errors.	SO3

2.8.3 Access control matrix

The SRS defines a single actor, so access control distinguishes an authenticated Inventory User from an unauthenticated person at the console.

Operation	Authenticated Inventory User	Unauthenticated
Login / Exit	Yes	Yes
Create initial account	No	First run only
Add / update / remove product	Yes	No
Search / view product, list all	Yes	No
Stock in / stock out	Yes	No
Check stock	Yes	No
View stock history	Yes	No
Inventory summary	Yes	No

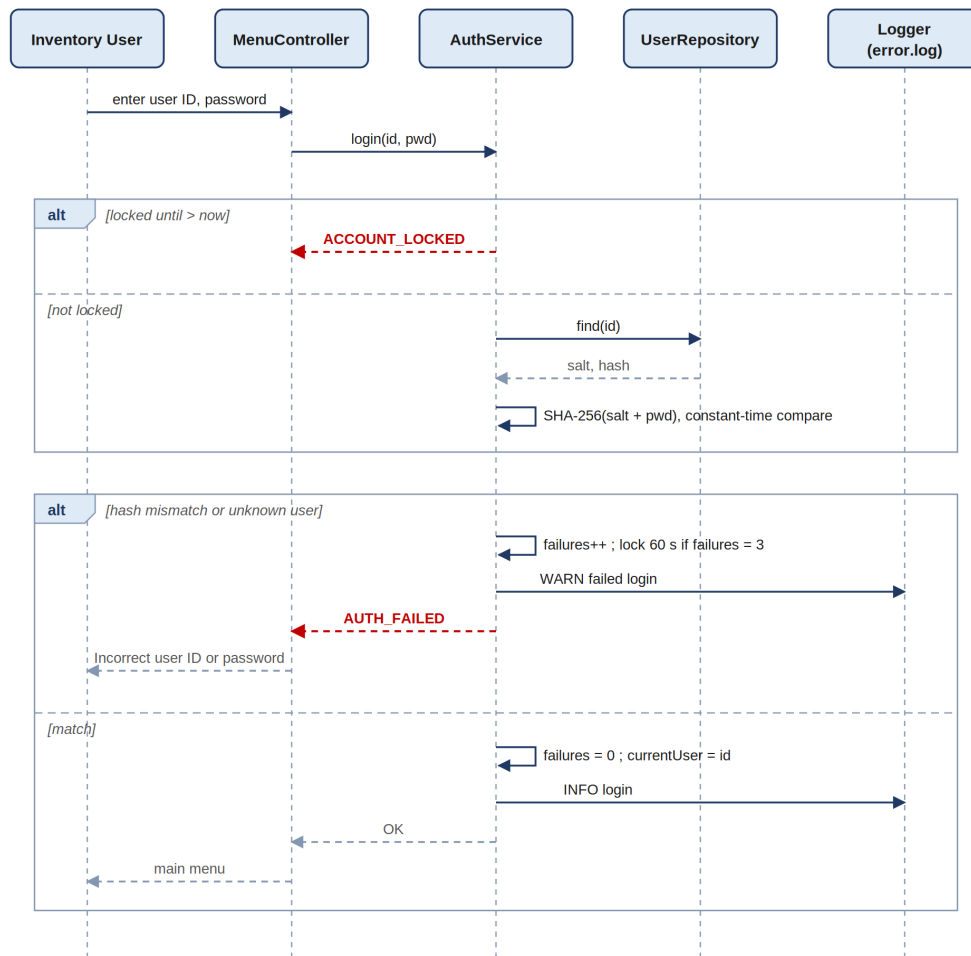
3. Detailed Design

3.1 UML Sequence Diagrams

Three interaction scenarios are modelled: **SD1 – User Login (UC01)**, **SD2 – Add Product (UC02)** and **SD3 – Stock-out with Ledger Entry (UC07)**. Solid arrows are calls, dashed arrows are returns and red arrows are error returns.

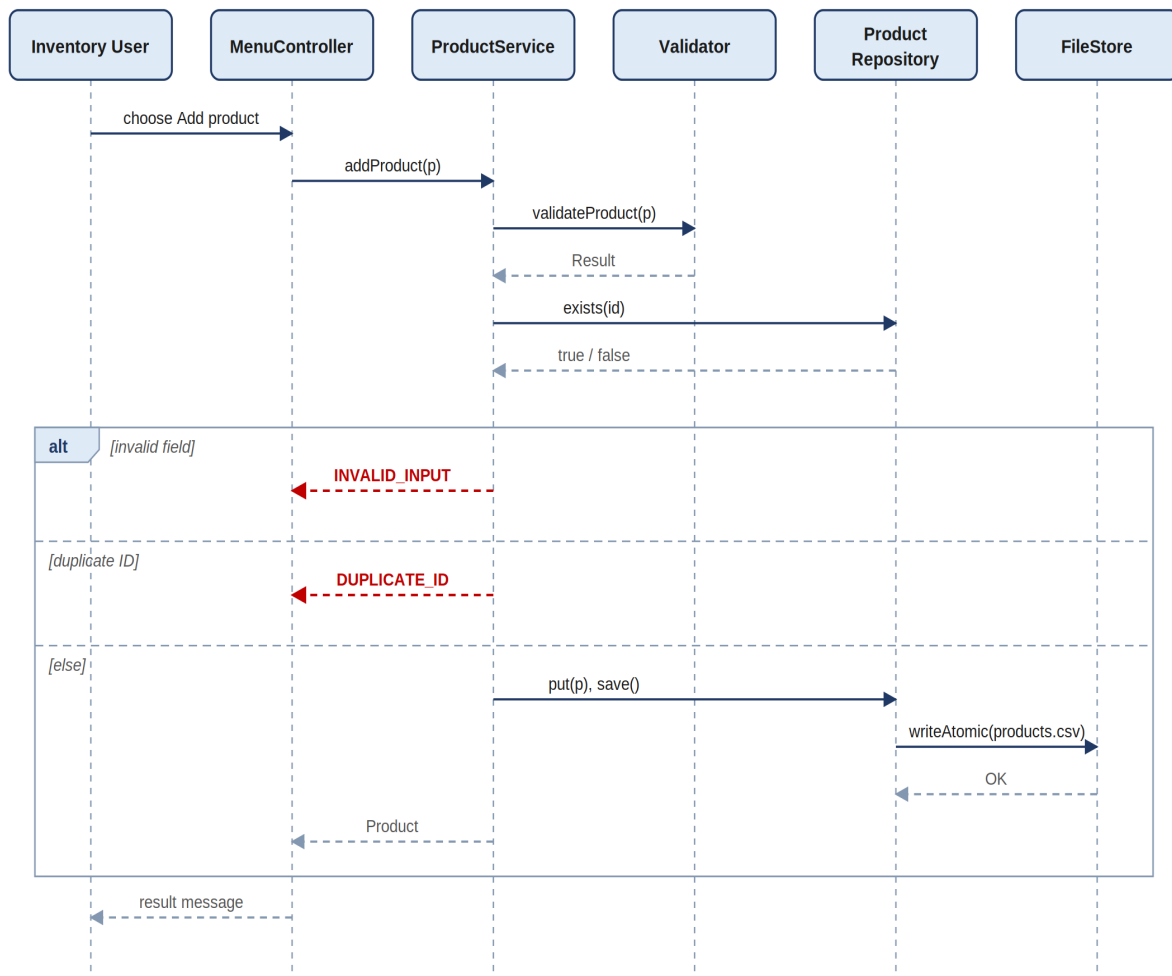
3.1.1 SD1 – User Login (UC01)

SD1 - User login with logout (UC01)



3.1.2 SD2 – Add Product (UC02)

SD2 - Add product (UC02)



Validation runs first, then the duplicate check against the in-memory map. **INVALID_INPUT** (with the field name) takes precedence over **DUPLICATE_ID**. Only a valid, new product reaches `save()`.

3.1.3 SD3 – Stock-out with Ledger Entry (UC07)

SD3 - Stock-out with ledger entry (UC07)

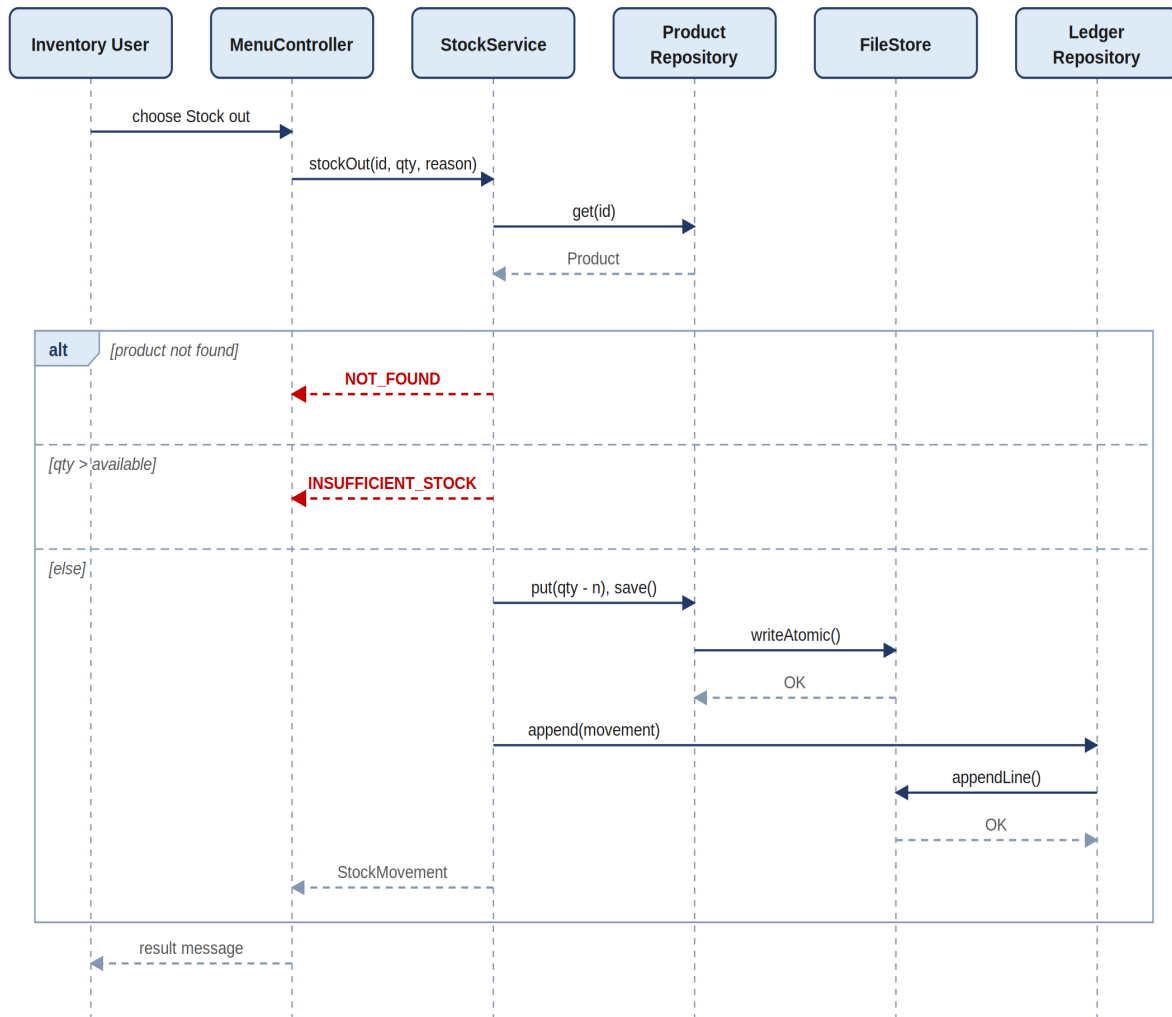


Figure 4: Sequence diagram for stock-out and ledger logging (FR07, FR09, VFR06, VFR07)

The product file is rewritten atomically before the movement is appended. If the append fails, the quantity is restored and the product file re-saved (Section 3.3.1).

3.2 API Design

RISMS has no network API. Its API is the set of C++ class interfaces between layers, plus the console menu that users interact with.

3.2.1 Conventions

- Every service operation returns **Result<T>**, carrying either a value or an ErrorCode with a human-readable message; expected failures never throw.
- Identifiers are std::string; quantities are int; prices are double rounded to two decimals; timestamps are ISO-8601 local time from the system clock.
- Services validate all arguments themselves, even when ConsoleUI has already checked them.

- Quantity is changed only through stockIn()/stockOut(), never through updateProduct(), so every change reaches the ledger.

3.2.2 Service operation summary

Operation	Purpose	Returns	Main errors
AuthService::login(id, pwd)	Authenticate and open a session	Result<void>	AUTH_FAILED, ACCOUNT_LOCKED
AuthService::logout()	End session, log event	void	—
ProductService::addProduct(p)	Create product record	Result<Product>	INVALID_INPUT, DUPLICATE_ID, FILE_IO_ERROR
ProductService::updateProduct(id, name, category, price)	Modify name, category or price	Result<Product>	NOT_FOUND, INVALID_INPUT, FILE_IO_ERROR
ProductService::removeProduct(id)	Remove product (history kept)	Result<void>	NOT_FOUND, FILE_IO_ERROR
ProductService::findById(id)	Exact lookup	Result<Product>	NOT_FOUND
ProductService::searchByName(text)	Case-insensitive substring search	Result<vector<Product>>	NOT_FOUND
ProductService::listAll()	All products sorted by ID	vector<Product>	—
StockService::stockIn(id, qty, reason)	Increase quantity, log movement	Result<StockMovement>	NOT_FOUND, INVALID_INPUT, FILE_IO_ERROR
StockService::stockOut(id, qty, reason)	Decrease quantity, log movement	Result<StockMovement>	NOT_FOUND, INVALID_INPUT, INSUFFICIENT_STOCK
StockService::getQuantity(id)	Current available quantity	Result<int>	NOT_FOUND
ReportService::historyFor(id)	Movements for a product, newest first	Result<vector<StockMovement>>	NOT_FOUND
ReportService::inventorySummary()	Totals and low-stock list	InventorySummary	—

3.2.3 Common types

```
enum class ErrorCode { OK, INVALID_INPUT, DUPLICATE_ID, NOT_FOUND,
    INSUFFICIENT_STOCK,
    AUTH_FAILED, ACCOUNT_LOCKED, NOT_AUTHENTICATED,
    FILE_IO_ERROR, DATA_CORRUPT };

template <typename T> struct Result {
    ErrorCode code = ErrorCode::OK;
    std::string message;           // shown to the user by ConsoleUI
    std::optional<T> value;
    bool ok() const { return code == ErrorCode::OK; }
};

enum class AdjustType { IN, OUT };
struct Product { std::string id, name, category; double unitPrice; int quantity; };
struct StockMovement { long id; std::string productId; AdjustType type;
    int change, resultingQty; std::string timestamp, userId,
    reason; };
struct InventorySummary { int totalProducts; long totalUnits; double totalValue;
    std::vector<Product> lowStock; };
```

3.2.5 Console menu (user-facing interface)

Key	Menu option	Service call	Use case
1	Add product	addProduct()	UC02
2	Update product	updateProduct()	UC03
3	Remove product (asks Y/N)	removeProduct()	UC04
4	Search product (by ID or name)	findById() / searchByName()	UC05, UC06
5	Stock in	stockIn()	UC07
6	Stock out	stockOut()	UC07
7	Check stock	getQuantity()	UC08
8	Stock history	historyFor()	UC09
9	List all products	listAll()	UC10
10	Inventory summary	inventorySummary()	UC11
0	Log out and exit	logout()	UC01

3.3 Error Handling

3.3.1 Strategy

- **Validate before mutate:** every service checks all arguments before changing any state (VFR02, SR04).
- **Typed error codes:** services return Result with an ErrorCode; they never print or exit themselves.
- **One place for presentation:** ConsoleUI turns each ErrorCode into the message in Section 3.3.3; details go only to error.log.
- **Consistency on failure:** a service updates the in-memory copy, calls save()/append(), and on failure restores the previous value before returning FILE_IO_ERROR. For stock adjustments the product file is saved first, then the ledger entry is appended; if the append fails the quantity is restored and the product file re-saved (VFR07, NFR02).
- **Corrupt data at start-up:** a line that fails to parse is skipped, its line number logged, and the user warned once; loading continues.
- **Unexpected errors:** a top-level try/catch in main() logs the exception and exits cleanly without writing partial data.

3.3.2 Standard error presentation

```
[ERROR] INSUFFICIENT_STOCK
Only 12 units of P0001 are available; cannot remove 20.
Press Enter to return to the menu.

error.log:
2026-09-28T14:07:40 WARN StockService stockOut P0001 qty=20 available=12 user=staff01
```

3.3.3 Error catalogue

Code	Raised when	User-facing message	Recovery	Related requirement
INVALID_INPUT	Empty field, negative price or quantity, non-numeric or over-length input	Invalid <field>: <reason>.	Re-prompt the field	VFR02, SR04

Code	Raised when	User-facing message	Recovery	Related requirement
DUPLICATE_ID	Product ID already stored	Product ID <id> already exists.	Ask for another ID	VFR01
NOT_FOUND	No product matches the key	No product found for '<key>'.	Return to menu	VFR04
INSUFFICIENT_STOCK	Stock-out quantity exceeds available	Only <n> units available; cannot remove <m>.	Return to menu, quantity unchanged	VFR06
AUTH_FAILED	Wrong user ID or password	Incorrect user ID or password.	Re-prompt; count attempt	FR01, SR01
ACCOUNT_LOCKED	3 consecutive failed logins	Too many attempts. Try again in 60 seconds.	Wait, then re-prompt	SR02
NOT_AUTHENTICATED	Command without a session	Please log in first.	Show login screen	SR03
FILE_IO_ERROR	File cannot be opened, written or renamed	Could not save data. Your last change was not applied.	Roll back, log, return to menu	NFR02, SR06
DATA_CORRUPT	A stored line fails to parse at load	Warning: <n> unreadable record(s) skipped (see error.log).	Skip line, keep loading	FR14, VFR08

3.4 Key Algorithms

3.4.1 Stock-out with ledger entry (FR07, FR09, VFR06, VFR07)

```

stockOut(id, qty, reason):
    if qty <= 0: return INVALID_INPUT
    p = products.get(id); if p is none: return NOT_FOUND
    if qty > p.quantity: return INSUFFICIENT_STOCK
    old = p.quantity
    p.quantity = old - qty; products.put(p)
    if products.save() fails: p.quantity = old; products.put(p); return FILE_IO_ERROR
    m = StockMovement(ledger.nextId(), id, OUT, qty, p.quantity, now(),
auth.currentUser(), reason)
    if ledger.append(m) fails:
        p.quantity = old; products.put(p); products.save(); return FILE_IO_ERROR
    return m

```

stockIn() is identical except that it has no availability check and sets new quantity = old + qty (VFR05).

3.4.2 Atomic file write (NFR02, SR06)

```

writeAtomic(path, lines):
    tmp = path + ".tmp"
    write all lines to tmp; flush; close
    if any stream error: delete tmp; return FILE_IO_ERROR
    std::filesystem::rename(tmp, path) // replaces the original in one step

```

3.4.3 Login with lockout (FR01, SR01, SR02)

```

login(id, pwd):
    if now() < lockUntil: return ACCOUNT_LOCKED
    u = users.find(id)
    ok = u exists and constantTimeEqual(sha256(u.salt + pwd), u.hash)
    if not ok:
        failures += 1; log WARN
        if failures >= maxAttempts: lockUntil = now() + lockoutSeconds; failures = 0
        return AUTH_FAILED
    failures = 0; current = id; log INFO; return OK

```

4. Design Constraints and Assumptions

- The system is a single-user C++17 console application running on one store PC; only one instance runs against a data/ directory at a time.
- Persistence uses files only, as required by the SRS; no external database or network service is used.
- Only free tools are used (g++/MSVC Community, CMake), in line with the economic feasibility in the SRS.
- The SRS defines one actor, the Inventory User; there are no separate administrator privileges in Part-1.
- The system clock of the store PC is the single time source for ledger timestamps.
- The low-stock threshold is one store-wide value in config.ini (default 10 units).